```
# 1. Install all required libraries
!pip install torch transformers datasets tokenizers numpy tqdm
```

```
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.9
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->to
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.1
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->t
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformer
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->data
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.13
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2026.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->f
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspe
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->f
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->hugg
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datase
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->hugg
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->hu
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0
```

```
# STEP 2 - LOAD REAL DATASETS (combine all three)
from datasets import load_dataset
import os

all_texts = []

try:
    print("Loading Wikipedia (via wikimedia/wikipedia)...")
    # Dataset 1: Wikipedia (English) - Using the 2023 snapshot available on the Hub
    wiki = load_dataset("wikimedia/wikipedia", "20231101.en", split="train[:5%]")
    all_texts.extend(wiki["text"])
    print(f"Wiki loaded: {len(wiki)} documents")

    print("Loading BookCorpus...")
    # Dataset 2: BookCorpus
    books = load_dataset("bookcorpus", split="train[:5%]")
    all_texts.extend(books["text"])
    print(f"Books loaded: {len(books)} documents")
```

```
    print("Loading OpenWebText...")
    # Dataset 3: OpenWebText
    web = load_dataset("Skylion007/openwebtext", split="train[:3%]")
    all_texts.extend(web["text"])
    print(f"Web loaded: {len(web)} documents")

except Exception as e:
    print(f"An error occurred while loading datasets: {e}")

# Combine and report stats
total_docs = len(all_texts)
total_chars = sum(len(t) for t in all_texts)

print(f"\n--- Dataset Summary ---")
print(f"Total documents: {total_docs:,}")
print(f"Total character count: {total_chars:,}")
```

```
# STEP 3 - TRAIN A REAL BPE TOKENIZER
from tokenizers import ByteLevelBPETokenizer
import os

# Initialize the tokenizer
tokenizer = ByteLevelBPETokenizer()

# Create a directory for the tokenizer
os.makedirs("my_tokenizer", exist_ok=True)

# Train the tokenizer
tokenizer.train_from_iterator(
    all_texts,
    vocab_size=16000,
    min_frequency=2,
    special_tokens=["<pad>", "<unk>", "<bos>", "<eos>"]
)

# Save the tokenizer
tokenizer.save_model("my_tokenizer")
print(f"Tokenizer trained. Vocabulary size: {tokenizer.get_vocab_size()}")
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_715/2323921193.py in <cell line: 0>()
     10
     11 # Train the tokenizer
---> 12 tokenizer.train_from_iterator(
     13     all_texts,
     14     vocab_size=16000,

/usr/local/lib/python3.12/dist-packages/tokenizers/implementations/byte_level_bpe.py in train_from_iterator(self, iterator,
vocab_size, min_frequency, show_progress, special_tokens, length)
    116             initial_alphabet=pre_tokenizers.ByteLevel.alphabet(),
    117         )
--> 118         self._tokenizer.train_from_iterator(
    119             iterator,
    120             trainer=trainer,

KeyboardInterrupt:
```

```
# STEP 4 - TOKENIZE AND PREPARE THE DATASET
import torch
import numpy as np
from torch.utils.data import DataLoader, TensorDataset

print("Tokenizing all texts...")
# Tokenize and flatten into one large list of IDs
full_token_ids = []
for text in all_texts:
    enc = tokenizer.encode(text)
    full_token_ids.extend(enc.ids)

# Convert to numpy for faster processing
full_token_ids = np.array(full_token_ids)

# Create sliding window chunks
max_len = 512
stride = 256
chunks = []

for i in range(0, len(full_token_ids) - max_len, stride):
```

```
    chunks.append(full_token_ids[i : i + max_len])

# Convert to torch tensors
tokens_tensor = torch.tensor(np.array(chunks), dtype=torch.long)
print(f"Created {tokens_tensor.shape[0]} chunks of length {max_len}.")

# Split into 90% train and 10% validation
train_size = int(0.9 * len(tokens_tensor))
val_size = len(tokens_tensor) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(tokens_tensor, [train_size, val_size])

print(f"Train dataset size: {len(train_dataset)}")
print(f"Validation dataset size: {len(val_dataset)}")
```

```
# 2. Download and prepare the wikitext-2-raw-v1 dataset
from datasets import load_dataset

dataset = load_dataset('wikitext', 'wikitext-2-raw-v1')
train_texts = [x['text'] for x in dataset['train'] if len(x['text']) > 0]
val_texts = [x['text'] for x in dataset['validation'] if len(x['text']) > 0]

print(f"Loaded {len(train_texts)} training samples.")
```

```
README.md:        10.5k/? [00:00<00:00, 636kB/s]
wikitext-2-raw-v1/test-00000-of-00001.pa(…): 100%  733k/733k [00:00<00:00, 3.65MB/s]
wikitext-2-raw-v1/train-00000-of-00001.p(…): 100%  6.36M/6.36M [00:00<00:00, 31.6MB/s]
wikitext-2-raw-v1/validation-00000-of-00(…): 100%  657k/657k [00:00<00:00, 2.64MB/s]
Generating test split: 100%  4358/4358 [00:00<00:00, 155265.42 examples/s]
Generating train split: 100%  36718/36718 [00:00<00:00, 452781.71 examples/s]
Generating validation split: 100%  3760/3760 [00:00<00:00, 132889.96 examples/s]
Loaded 23767 training samples.
```

```
# 3. Train a BPE tokenizer
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
trainer = BpeTrainer(vocab_size=8000, special_tokens=["[UNK]", "[CLS]", "[SEP]", "[PAD]", "[MASK]"])
tokenizer.pre_tokenizer = Whitespace()

# Train the tokenizer on the training text
tokenizer.train_from_iterator(train_texts, trainer)
tokenizer.save("tokenizer.json")
print("Tokenizer trained and saved.")
```

```
Tokenizer trained and saved.
```

```
# 4. Build a GPT-style Transformer model in PyTorch
import torch
import torch.nn as nn
import math

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class SLMConfig:
    vocab_size = 8000
    emb_dim = 256
    n_layers = 4
    n_heads = 4
    ff_dim = 1024
    max_len = 256
    dropout = 0.1

class MultiHeadAttention(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.n_heads = cfg.n_heads
        self.head_dim = cfg.emb_dim // cfg.n_heads
        self.wq = nn.Linear(cfg.emb_dim, cfg.emb_dim)
        self.wk = nn.Linear(cfg.emb_dim, cfg.emb_dim)
        self.wv = nn.Linear(cfg.emb_dim, cfg.emb_dim)
        self.wo = nn.Linear(cfg.emb_dim, cfg.emb_dim)
```

```
        self.dropout = nn.Dropout(cfg.dropout)

    def forward(self, x, mask=None):
        b, t, c = x.size()
        q = self.wq(x).view(b, t, self.n_heads, self.head_dim).transpose(1, 2)
        k = self.wk(x).view(b, t, self.n_heads, self.head_dim).transpose(1, 2)
        v = self.wv(x).view(b, t, self.n_heads, self.head_dim).transpose(1, 2)

        attn = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)
        if mask is not None:
            attn = attn.masked_fill(mask == 0, float('-inf'))
        attn = torch.softmax(attn, dim=-1)
        attn = self.dropout(attn)

        out = (attn @ v).transpose(1, 2).contiguous().view(b, t, c)
        return self.wo(out)

class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.ln1 = nn.LayerNorm(cfg.emb_dim)
        self.attn = MultiHeadAttention(cfg)
        self.ln2 = nn.LayerNorm(cfg.emb_dim)
        self.ff = nn.Sequential(
            nn.Linear(cfg.emb_dim, cfg.ff_dim),
            nn.GELU(),
            nn.Linear(cfg.ff_dim, cfg.emb_dim),
            nn.Dropout(cfg.dropout)
        )

    def forward(self, x, mask=None):
        x = x + self.attn(self.ln1(x), mask)
        x = x + self.ff(self.ln2(x))
        return x

class SLM(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.cfg = cfg
        self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.emb_dim)
        self.pos_emb = nn.Parameter(torch.zeros(1, cfg.max_len, cfg.emb_dim))
        self.blocks = nn.ModuleList([TransformerBlock(cfg) for _ in range(cfg.n_layers)])
        self.ln_f = nn.LayerNorm(cfg.emb_dim)
        self.head = nn.Linear(cfg.emb_dim, cfg.vocab_size, bias=False)

    def forward(self, idx, targets=None):
        b, t = idx.size()
        mask = torch.tril(torch.ones(t, t)).to(device)
        x = self.tok_emb(idx) + self.pos_emb[:, :t, :]
        for block in self.blocks:
            x = block(x, mask)
        logits = self.head(self.ln_f(x))

        loss = None
        if targets is not None:
            loss = nn.functional.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
        return logits, loss

cfg = SLMConfig()
model = SLM(cfg).to(device)
print(f"Total parameters: {sum(p.numel() for p in model.parameters()):,}")
```

```
Total parameters: 7,321,088
```

```
# 5. Efficient DataLoader
import numpy as np

def get_batches(data, batch_size, seq_len):
    # Flatten data and chunk it
    n_batches = len(data) // (batch_size * seq_len)
    data = data[:n_batches * batch_size * seq_len]
    data = data.reshape(batch_size, -1)
    for i in range(0, data.shape[1] - seq_len, seq_len):
        x = data[:, i:i+seq_len]
        y = data[:, i+1:i+seq_len+1]
        yield torch.tensor(x, dtype=torch.long).to(device), torch.tensor(y, dtype=torch.long).to(device)

# Tokenize full dataset once
all_tokens = []
for text in train_texts:
```

```
    all_tokens.extend(tokenizer.encode(text).ids)
train_data = np.array(all_tokens)
```

```
# 6. Full Training Loop
from torch.optim import AdamW
from tqdm import tqdm

optimizer = AdamW(model.parameters(), lr=3e-4)
model.train()

epochs = 3
step = 0
for epoch in range(epochs):
    batch_gen = get_batches(train_data, batch_size=32, seq_len=cfg.max_len)
    for x, y in batch_gen:
        logits, loss = model(x, y)

        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        if step % 100 == 0:
            print(f"Epoch {epoch+1} | Step {step} | Loss: {loss.item():.4f}")
        step += 1
```

```
Epoch 1 | Step 0 | Loss: 9.1509
Epoch 1 | Step 100 | Loss: 6.6833
Epoch 1 | Step 200 | Loss: 6.3459
Epoch 1 | Step 300 | Loss: 6.1894
Epoch 2 | Step 400 | Loss: 5.9035
Epoch 2 | Step 500 | Loss: 5.7389
Epoch 2 | Step 600 | Loss: 5.6575
Epoch 3 | Step 700 | Loss: 5.5540
Epoch 3 | Step 800 | Loss: 5.5254
Epoch 3 | Step 900 | Loss: 5.3988
```

```
# 7. Text Generation Function
@torch.no_grad()
def generate(model, tokenizer, prompt, max_new_tokens=100, k=10):
    model.eval()
    ids = torch.tensor(tokenizer.encode(prompt).ids, dtype=torch.long).unsqueeze(0).to(device)

    for _ in range(max_new_tokens):
        # Crop context if needed
        idx_cond = ids[:, -cfg.max_len:]
        logits, _ = model(idx_cond)
        logits = logits[:, -1, :] # Get last token logits

        # Top-k sampling
        v, _ = torch.topk(logits, k)
        logits[logits < v[:, [-1]]] = -float('Inf')
        probs = torch.softmax(logits, dim=-1)

        next_id = torch.multinomial(probs, num_samples=1)
        ids = torch.cat((ids, next_id), dim=1)

    return tokenizer.decode(ids[0].tolist())
```

```
# 8. Test the model
prompt = "The history of science"
generated_text = generate(model, tokenizer, prompt)
print(f"Prompt: {prompt}\nGenerated: {generated_text}")
```

```
Prompt: The history of science
Generated: The history of science fiction ( a , and is a large , and the same year . The first , the main characters and has not
```

```
# 9. Save and Download model
torch.save(model.state_dict(), 'slm_model.pt')
from google.colab import files
# files.download('slm_model.pt') # Uncomment to trigger browser download
```

Start coding or generate with AI.

```
# 10. Generate text with a different prompt
new_prompt = "In the distant future, artificial intelligence"
new_generated_text = generate(model, tokenizer, new_prompt, max_new_tokens=100)
```

```
print(f"Prompt: {new_prompt}\nGenerated: {new_generated_text}")
```

```
Prompt: In the distant future, artificial intelligence
Generated: In the dist ant future , art ificial intelligence as the first to take a large @-@ up of the second , and the city of
```

```
new_prompt = "what is artificial intelligence"
new_generated_text = generate(model, tokenizer, new_prompt, max_new_tokens=100)
print(f"Prompt: {new_prompt}\nGenerated: {new_generated_text}")
```

```
Prompt: what is artificial intelligence
Generated: what is art ificial intelligence as an im pression . A . The two common to have been suggested that they are not to t
```